ROZDZIAŁ 3.

Klasyfikacja

W rozdziale 1. wspomniałem, że najpowszechniej stosowanymi metodami uczenia nadzorowanego są regresja (przewidywanie wartości) i klasyfikacja (przewidywanie klas). Rozdział 2. poświęciłem omówieniu mechanizmu regresji oraz prognozowania cen mieszkań za pomocą różnych algorytmów, takich jak regresja liniowa, drzewa decyzyjne czy losowy las (przyjrzymy im się dokładniej w następnych rozdziałach). Teraz skoncentrujemy się na modelach klasyfikacji.

Zbiór danych MNIST

W niniejszym rozdziale będziemy korzystać ze zbioru danych MNIST, składającego się z 70 000 małych rysunków zawierających cyfry odręcznie zapisane przez uczniów szkół średnich i pracowników amerykańskiego Biura Spisu Ludności (ang. *US Census Bureau*). Każdy rysunek zawiera etykietę określającą zawartą na nim cyfrę. Omawiany zestaw danych został tak dogłębnie przeanalizowany, że w świecie uczenia maszynowego bywa określany mianem powitalnego zbioru danych dla nowych adeptów: po napisaniu nowego algorytmu klasyfikującego jest on najpierw sprawdzany właśnie na zbiorze MNIST, a każda osoba poznająca meandry uczenia maszynowego prędzej czy później musi na niego natrafić.

Moduł Scikit-Learn zawiera wiele funkcji ułatwiających pobieranie popularnych zestawów danych. Jak łatwo zgadnąć, MNIST stanowi jeden z takich zbiorów. Możemy go pobrać za pomocą poniższego kodu[1]:

```
>>> from sklearn.datasets import fetch_openml
>>> mnist = fetch_openml('mnist_784', version=1)
>>> mnist.keys()
dict_keys(['data', 'target', 'feature_names', 'DESCR', 'details',
'categories', 'url'])
```

Zazwyczaj zbiory danych wczytywane przez moduł Scikit-Learn mają podobną strukturę składającą się z następujących elementów:

- klucza `DESCR` opisującego zbiór danych,
- klucza `data` zawierającego tablicę, w której każdy wiersz reprezentuje przykład, a kolumna — cechę,
- klucza `target` przechowującego tablicę etykiet.

[1] Domyślnie moduł Scikit-Learn przechowuje pobrane zbiory danych w katalogu *$HOME/scikit_learn_data*.

Przyjrzyjmy się teraz tym tablicom:

```
>>> X, y = mnist["data"], mnist["target"]
>>> X.shape
(70000, 784)
>>> y.shape
(70000,)
```

W tym zbiorze danych występuje 70 000 obrazów, a każdy z nich jest opisany 784 cechami. Wynika to z faktu, że obraz ma rozmiar 28×28 pikseli i każda cecha opisuje natężenie szarości danego piksela i przyjmuje wartości od 0 (kolor biały) do 255 (kolor czarny). Sprawdźmy, jak wygląda przykładowa cyfra przechowywana w zestawie danych. Wystarczy w tym celu wybrać wektor cech danej próbki i przekształcić go w macierz o rozmiarze 28×28, którą następnie możemy wyświetlić za pomocą funkcji `imshow()` modułu Matplotlib:

```
import matplotlib as mpl
import matplotlib.pyplot as plt

some_digit = X[0]
some_digit_image = some_digit.reshape(28, 28)

plt.imshow(some_digit_image, cmap = "binary")
plt.axis("off")
plt.show()
```



Obraz ten przypomina z kształtu cyfrę 5 i, zgodnie z etykietą, tak jest w istocie:

```
>>> y[0]
'5'
```

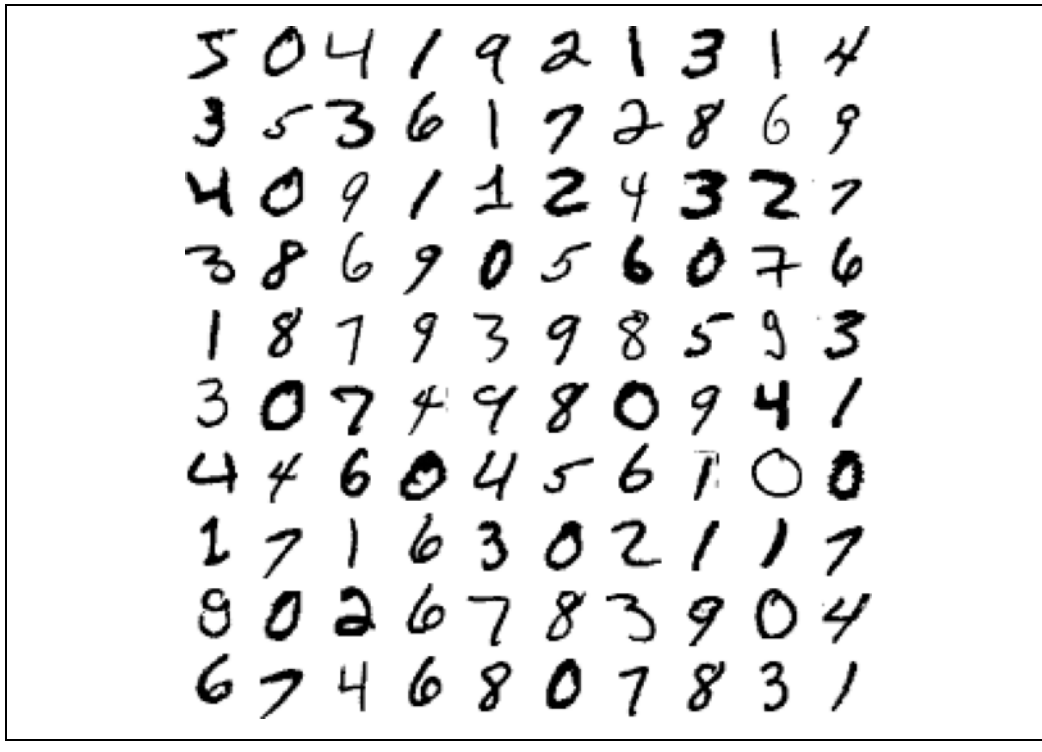
Zwróć uwagę, że etykieta jest łańcuchem znaków. Większość algorytmów uczenia maszynowego spodziewa się wartości numerycznych, dlatego przekształćmy y w liczbę całkowitą:

```
>>> y = y.astype(np.uint8)
```

Na rysunku 3.1 pokazuję kilka innych obrazów ze zbioru danych MNIST, dzięki czemu możesz sobie uświadomić złożoność zadania klasyfikującego.

Chwileczkę! Przed dokładniejszym przyjrzeniem się danym powinniśmy zawsze najpierw stworzyć zbiór testowy i odstawić go na bok. W rzeczywistości zestaw MNIST jest już podzielony na zbiory uczący (pierwsze 60 000 przykładów) i testowy (pozostałe 10 000 obrazów):

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```



Rysunek 3.1. *Cyfry ze zbioru danych MNIST*

Zestaw danych uczących został już za nas przetasowany, co jest dobrym rozwiązaniem, gdyż gwarantuje, że podzbiory utworzone podczas sprawdzianu krzyżowego będą do siebie podobne (nie chcemy, aby w którymś podzbiorze zabrakło jakiejś cyfry). Ponadto niektóre algorytmy są wrażliwe na kolejność próbek uczących i nie za dobrze sobie radzą, jeżeli przetwarzają wiele podobnych przykładów z rzędu. Unikniemy tego problemu dzięki przetasowaniu zbioru danych[2].

Uczenie klasyfikatora binarnego

Uprośćmy na razie problem i spróbujmy identyfikować tylko jedną cyfrę, np. 5. Ten „wykrywacz piątek" będzie stanowił przykład **klasyfikatora binarnego** (ang. *binary classifier*), zdolnego do rozpoznawania jedynie dwóch klas: piątek i niepiątek. Stwórzmy wektory docelowe dla tego zadania klasyfikującego:

```
y_train_5 = (y_train == 5) # Wartość True dla piątek, False dla wszystkich pozostałych cyfr
y_test_5 = (y_test == 5)
```

Teraz wybierzmy i wytrenujmy jakiś klasyfikator. Warto rozpocząć od klasyfikatora **stochastycznego spadku wzdłuż gradientu** (ang. *Stochastic Gradient Descent* — SGD) — posłuży nam do tego klasa `SGDClassifier`. Algorytm ten cechuje się możliwością wydajnego przetwarzania bardzo dużych zesta-

[2] W pewnych sytuacjach tasowanie danych może być złym pomysłem — np. w czasie pracy z danymi szeregu czasowego (takimi jak ceny akcji giełdowych czy warunki meteorologiczne). Poruszymy to zagadnienie w dalszej części książki.

wów danych. Wynika to częściowo z faktu, że klasyfikator SGD przetwarza poszczególne przykłady uczące niezależnie od siebie, po jednym naraz (z tego powodu nadaje się on również do **uczenia przyrostowego**), o czym przekonamy się już niebawem. Stwórzmy klasyfikator `SGDClassifier` i wytrenujmy go wobec całego zbioru uczącego:

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```



Skuteczność klasyfikatora `SGDClassifier` zależy od losowości na etapie uczenia (stąd przymiotnik „stochastyczny"). Jeżeli chcesz, aby wyniki były powtarzalne, wyznacz wartość parametru `random_state`.

Możemy teraz posłużyć się tym algorytmem do wykrywania cyfry 5 w zbiorze danych:

```
>>> sgd_clf.predict([some_digit])
array([True])
```

Klasyfikator zgaduje, że na wyznaczonym obrazie znajduje się cyfra 5 (wartość `True`). Wygląda na to, że się nie pomylił w tym konkretnym przypadku! Oceńmy teraz wydajność modelu.

Miary wydajności

Często ocena klasyfikatora stanowi większe wyzwanie od ewaluacji regresora, dlatego poświęcimy temu zagadnieniu dużą część niniejszego rozdziału. Istnieje wiele miar wydajności, dlatego zaparz sobie następną kawę i przygotuj się na kolejną porcję nowych pojęć i skrótów!

Pomiar dokładności za pomocą sprawdzianu krzyżowego

Dobrym sposobem oceny modelu jest zastosowanie użytej już przez nas w rozdziale 2. metody sprawdzianu krzyżowego.

Wykorzystajmy funkcję `cross_val_score()` do oceny naszego modelu `SDGClassifier` za pomocą metody sprawdzianu krzyżowego; wygenerujemy trzy podzbiory. Przypominam, że kroswalidacja k-krotna oznacza rozdzielenie zestawu uczącego na k podzbiorów (w tym przypadku k = 3), następnie wyuczenie modelu na k–1 podzbiorach, a na końcu przeprowadzenie prognoz i ich ocena wobec ostatniego podzbioru, a wszystko powtórzone k razy, tak że zostają wykorzystane wszystkie podzbiory (zostało to dokładniej wyjaśnione w rozdziale 2.):

```
>>> from sklearn.model_selection import cross_val_score
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.96355, 0.93795, 0.95615])
```

O rety! **Dokładność** powyżej 93% (współczynnik prawidłowych prognoz) wobec wszystkich podzbiorów sprawdzianu krzyżowego? Zdumiewające, nieprawdaż? Zanim się zanadto podekscytujemy, przyjrzyjmy się bardzo prostemu klasyfikatorowi klasyfikującemu jedynie obrazy niebędące piątkami:

Implementacja sprawdzianu krzyżowego

Od czasu do czasu będziemy potrzebować większej kontroli nad metodą kroswalidacji, niż zapewnia nam domyślnie moduł Scikit-Learn. W takim przypadku możemy samodzielnie zaimplementować sprawdzian krzyżowy. Poniższy kod wykonuje mniej więcej te same operacje, co funkcja `cross_val_score()` i wyświetla te same wyniki:

```
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

skfolds = StratifiedKFold(n_splits=3, random_state=42)

for train_index, test_index in skfolds.split(X_train, y_train_5):
    clone_clf = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]
    X_test_fold = X_train[test_index]
    y_test_fold = y_train_5[test_index]

    clone_clf.fit(X_train_folds, y_train_folds)
    y_pred = clone_clf.predict(X_test_fold)
    n_correct = sum(y_pred == y_test_fold)
print(n_correct / len(y_pred)) # Wyświetla wyniki 0.9502, 0.96565 i 0.96495
```

Klasa `StratifiedKFold` przeprowadza losowanie warstwowe (zostało ono omówione w rozdziale 2.), dzięki któremu otrzymujemy podzbiory zawierające reprezentacyjne populacje każdej z klas. W każdym przebiegu kod tworzy klon klasyfikatora, trenuje go za pomocą podzbiorów uczących i ocenia zdolność prognostyczną na podzbiorze testowym. Następnie jest zliczana liczba prawidłowych prognoz i zostaje wyświetlony współczynnik prawidłowych prognoz.

```
from sklearn.base import BaseEstimator

class Never5Classifier(BaseEstimator):
    def fit(self, X, y=None):
        return self
    def predict(self, X):
        return np.zeros((len(X), 1), dtype=bool)
```

Domyślasz się, jaka będzie dokładność tego modelu? Sprawdźmy:

```
>>> never_5_clf = Never5Classifier()
>>> cross_val_score(never_5_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.91125, 0.90855, 0.90915])
```

Zgadza się, uzyskujemy dokładność rzędu 90%! Wynika to jedynie z faktu, że tylko około 10% wszystkich obrazów stanowią piątki, dlatego gdybyśmy zawsze zgadywali, że dany obraz nie jest piątką, to mielibyśmy rację w ok. 90% przypadków. Nostradamus nie umywa się do nas.

Teraz już wiemy, dlaczego generalnie dokładność nie stanowi dobrej miary wydajności klasyfikatorów, zwłaszcza w przypadku **wypaczonych zbiorów danych** (ang. *skewed datasets*; są to takie zbiory danych, w których niektóre klasy występują znacznie częściej od pozostałych).

Macierz pomyłek

Znacznie lepszym sposobem oceny wydajności klasyfikatora jest analiza **macierzy pomyłek** (ang. *confusion matrix*). Ogólna koncepcja polega tu na zliczaniu przypadków zaklasyfikowania próbek z klasy A jako przykładów należących do klasy B. Przykładowo aby dowiedzieć się, ile razy algorytm pomylił obrazy piątek z obrazami trójek, wystarczy spojrzeć na piąty rząd i trzecią kolumnę macierzy pomyłek.

W celu obliczenia macierzy pomyłek musimy najpierw uzyskać zbiór prognoz, które porównamy z rzeczywistymi wartościami docelowymi. Moglibyśmy prognozować wyniki dla zbioru testowego, jednak na razie nie będziemy go ruszać (pamiętaj, że chcemy wykorzystać zestaw testowy jedynie na samym końcu projektu, gdy nasz klasyfikator będzie już gotowy do pracy). Zamiast tego skorzystamy z funkcji `cross_val_predict()`:

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

Obydwie funkcje, `cross_val_score()` i `cross_val_predict()`, przeprowadzają k-krotny sprawdzian krzyżowy, jednak ta druga nie zwraca wyników ewaluacji, lecz prognozy uzyskane dla każdego podzbioru testowego. Oznacza to, że możemy otrzymać „czystą" prognozę dla każdej próbki stanowiącej część zbioru uczącego („czysta" w tym kontekście znaczy, że prognoza została uzyskana przez model, który wcześniej nie widział tego przykładu na etapie uczenia).

Teraz możemy uzyskać macierz pomyłek za pomocą funkcji `confusion_matrix()`. Wystarczy podać jej klasy docelowe (`y_train_5`) i przewidywane klasy (`y_train_pred`):

```
>>> from sklearn.metrics import confusion_matrix
>>> confusion_matrix(y_train_5, y_train_pred)
array([[53057, 1522],
       [1325, 4096]])
```

Każdy rząd w macierzy pomyłek reprezentuje **rzeczywistą klasę**, natomiast kolumna symbolizuje **przewidywaną klasę**. Pierwszy rząd w naszej macierzy przechowuje obrazy niebędące piątkami (**klasa negatywna**): 53 053 próbki zostały prawidłowo sklasyfikowane jako niebędące piątkami (są to tak zwane przykłady **prawdziwie negatywne** — PN), natomiast pozostałe 1522 obrazy zostały niewłaściwie uznane za piątki (**fałszywie pozytywne** — FP). W drugim rzędzie zostały umieszczone obrazy zaklasyfikowane jako piątki (**klasa pozytywna**): 1325 próbek jest nieprawidłowo sklasyfikowanych jako niebędące piątkami (**fałszywie negatywne** — FN), zaś pozostałe 4096 prawidłowo zostało rozpoznanych jako piątki (**prawdziwie pozytywne** — PP). Doskonały klasyfikator uzyskiwałby wyłącznie przykłady prawdziwie pozytywne i prawdziwie negatywne, zatem niezerowe wartości w macierzy pomyłek mieściłyby się jedynie w głównej przekątnej (od lewego górnego do prawego dolnego rogu):

```
>>> y_train_perfect_predictions = y_train_5 # Udajemy, że uzyskaliśmy perfekcyjne rezultaty

>>> confusion_matrix(y_train_5, y_train_perfect_predictions)
array([[54579, 0],
       [0, 5421]])
```

Macierz pomyłek dostarcza wielu informacji, czasami jednak przydaje się bardziej zwięzły wskaźnik. Interesującym rozwiązaniem okazuje się dokładność pozytywnych prognoz — jest to tak zwana **precyzja** klasyfikatora (równanie 3.1).

Równanie 3.1. Precyzja

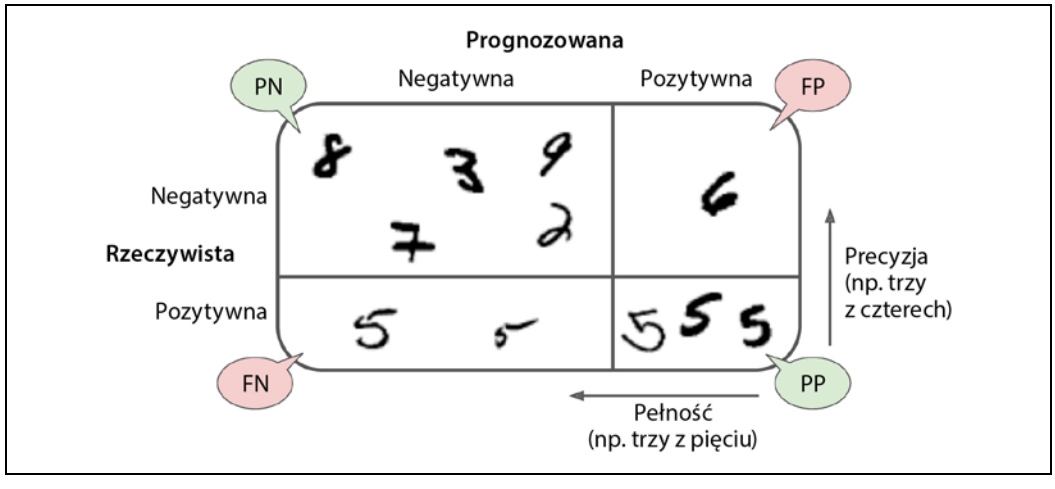
$$\text{Precyzja} = \frac{PP}{PP + FP}$$

Najbardziej banalnym sposobem uzyskania doskonałej precyzji jest otrzymanie pojedynczej pozytywnej prognozy i upewnienie się, że jest prawidłowa (Precyzja = 1/1 = 100%). Nie byłoby to jednak zbyt przydatne, ponieważ klasyfikator ignorowałby wszystkie próbki oprócz jednej pozytywnej. Dlatego zazwyczaj precyzja jest używana wraz z innym wskaźnikiem, zwanym **pełnością**, **czułością** lub **odsetkiem prawdziwie pozytywnych** (OPP): jest to odsetek pozytywnych przykładów, które zostały prawidłowo rozpoznane przez klasyfikator (równanie 3.2).

Równanie 3.2. Pełność

$$\text{Pełność} = \frac{PP}{PP + FN}$$

Jeśli nie do końca rozumiesz mechanizm działania macierzy pomyłek, rysunek 3.2 powinien pomóc rozwiać wątpliwości.



Rysunek 3.2. *Schemat macierzy pomyłek ukazujący przykłady prawdziwie negatywne (lewy górny róg), fałszywie pozytywne (prawy górny róg), fałszywie negatywne (lewy dolny róg) i prawdziwie pozytywne (prawy dolny róg)*

Precyzja i pełność

Moduł Scikit-Learn zawiera kilka funkcji obliczających wskaźniki klasyfikatorów, w tym również precyzję i pełność:

```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred) # == 4096 / (4096 + 1522)
0.7290850836596654
>>> recall_score(y_train_5, y_train_pred) # == 4096 / (4096 + 1325)
0.7555801512636044
```

Teraz nasz wykrywacz piątek nie jest już tak imponujący jak na etapie sprawdzania jego dokładności. Gdy uznaje, że rozpoznaje cyfrę 5, nie myli się jedynie w 72,9% przypadków, a do tego prawidłowo rozpoznaje jedynie 75,6% piątek.

Często wygodnie jest połączyć precyzję i pełność w jednym wskaźniku, zwanym **wynikiem F_1**, zwłaszcza jeżeli chcesz w prosty sposób porównać dwa klasyfikatory. Wynik F_1 stanowi **średnią harmoniczną** precyzji i pełności (równanie 3.3). Standardowa średnia traktuje wszystkie wartości jednakowo, natomiast średnia harmoniczna nadaje większą wagę małym wartościom. W rezultacie klasyfikator uzyska dużą wartość wyniku F_1 jedynie wtedy, gdy zarówno precyzja, jak i pełność będą miały dużą wartość.

Równanie 3.3. Wynik F_1

$$F_1 = \frac{2}{\frac{1}{\text{Precyzja}} + \frac{1}{\text{Pełność}}} = 2 \times \frac{\text{Precyzja} \times \text{Pełność}}{\text{Precyzja} + \text{Pełność}} = \frac{PP}{PP + \frac{FN + FP}{2}}$$

Aby obliczyć wynik F_1, wystarczy wywołać funkcję `f1_score()`:

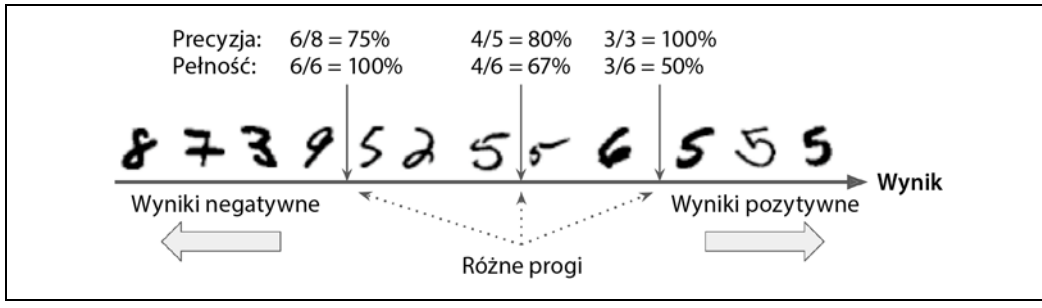
```
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7420962043663375
```

Wynik F_1 faworyzuje klasyfikatory mające zbliżone wartości precyzji i pełności. Nie zawsze tego chcemy: w pewnych sytuacjach zależy nam bardziej na precyzji, a w innych na pełności. Przykładowo, po wyuczeniu klasyfikatora w określaniu filmów bezpiecznych dla dzieci, prawdopodobnie wolelibyśmy klasyfikator odrzucający wiele dobrych filmów (mała wartość pełności), ale zapamiętujący jedynie bezpieczne (duża precyzja); bardzo niekorzystna byłaby sytuacja, w której model cechuje się znacznie większą pełnością, ale jednocześnie dopuszcza kilka nieodpowiednich filmów (w takim przypadku należałoby nawet dodać do potoku element ludzki — osobę, która sprawdzałaby wyniki klasyfikatora). Z drugiej strony, załóżmy, że trenujemy klasyfikator rozpoznający złodziei z zapisów kamer: prawdopodobnie nic się złego nie stanie, jeśli model będzie miał tylko 30% precyzji przy 99% pełności (jasne, ochroniarze wygenerują kilka fałszywych alarmów, ale niemal wszyscy przestępcy zostaną złapani).

Niestety nie możemy mieć wszystkiego naraz: wraz ze wzrostem precyzji maleje pełność i odwrotnie. Zależność ta jest nazywana **kompromisem pomiędzy precyzją a pełnością**.

Kompromis pomiędzy precyzją a pełnością

Aby zrozumieć ten kompromis, przyjrzyjmy się, w jaki sposób klasyfikator `SGDClassifier` podejmuje decyzje. Dla każdej próbki zostaje wyliczony wynik na podstawie **funkcji decyzyjnej**. Jeżeli jego wartość przekroczy określony próg, przykład ten zostanie przydzielony do klasy pozytywnej; w przeciwnym wypadku będzie wyznaczony do klasy negatywnej. Na rysunku 3.3 widzimy kilka cyfr umieszczonych w kolejności od najniższego do najwyższego wyniku. Załóżmy, że **próg decyzyjny** znajduje się w miejscu wskazywanym przez środkową strzałkę (pomiędzy dwiema piątkami): w ten sposób otrzymujemy po prawej stronie 4 próbki prawdziwie pozytywne (rzeczywiste piątki) i jedną fałszywie pozytywną (cyfra 6). Zatem przy tak dobranym progu precyzja wynosi 80% (cztery z pięciu). Jednak na sześć dostępnych piątek zostają rozpoznane tylko cztery, dlatego uzyskujemy tu pełność na poziomie 67% (cztery z sześciu). Jeśli podniesiemy próg (przesuniemy go w prawo), przykład fałszywie



Rysunek 3.3. *W powyższym kompromisie pomiędzy precyzją a pełnością obrazy są oceniane zgodnie z wynikiem klasyfikatora, a te, których wynik przekroczy wyznaczony próg decyzyjny, zostają uznane za pozytywne; im wyższy próg, tym mniejsza pełność, ale (zazwyczaj) większa precyzja*

pozytywny (cyfra 6) stanie się prawdziwie negatywny, dzięki czemu wzrośnie precyzja (do 100% w omawianym przykładzie), ale z kolei jedna wartość prawdziwie pozytywna przekształci się w fałszywie negatywną, przez co wartość pełności zmaleje do 50%. Z kolei obniżenie progu (przesunięcie go w lewo) zwiększy pełność przy jednoczesnej redukcji precyzji.

Moduł Scikit-Learn nie pozwala na bezpośrednie dobieranie progów, ale daje nam dostęp do wyników decyzyjnych, za pomocą których są wyliczane prognozy. Zamiast wywoływać metodę `predict()`, możemy skorzystać z metody `decision_function()`, która zwraca obliczony wynik dla każdej próbki, a następnie wyliczyć prognozy na podstawie tych wyników przy użyciu dowolnego progu:

```
>>> y_scores = sgd_clf.decision_function([some_digit])
>>> y_scores
array([2412.53175101])
>>> threshold = 0
>>> y_some_digit_pred = (y_scores > threshold)
array([True])
```

Klasyfikator `SGDClassifier` korzysta z progu o wartości 0, dlatego powyższy kod zwraca taki sam wynik, jak metoda `predict()` (np. `True`). Podnieśmy teraz ten próg.

```
>>> threshold = 8000
>>> y_some_digit_pred = (y_scores > threshold)
>>> y_some_digit_pred
array([False])
```

Uzyskaliśmy w ten sposób dowód, że podniesienie progu zmniejsza pełność. Analizowany obraz w rzeczywistości symbolizuje cyfrę 5 i klasyfikator rozpoznaje ją przy progu o wartości 0, ale myli się, gdy wartość progu zostanie podniesiona do wartości 8000.

W jaki sposób należy dobierać próg? Najpierw skorzystaj z funkcji cross_val_predict(), aby uzyskać wyniki wszystkich przykładów zestawu uczącego, tym razem jednak zaznacz, że chcesz otrzymać nie prognozy, lecz wyniki decyzyjne:

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,
                             method="decision_function")
```

Mając te wyniki, możesz wyliczyć precyzję i pełność dla wszystkich możliwych progów za pomocą funkcji `precision_recall_curve()`:

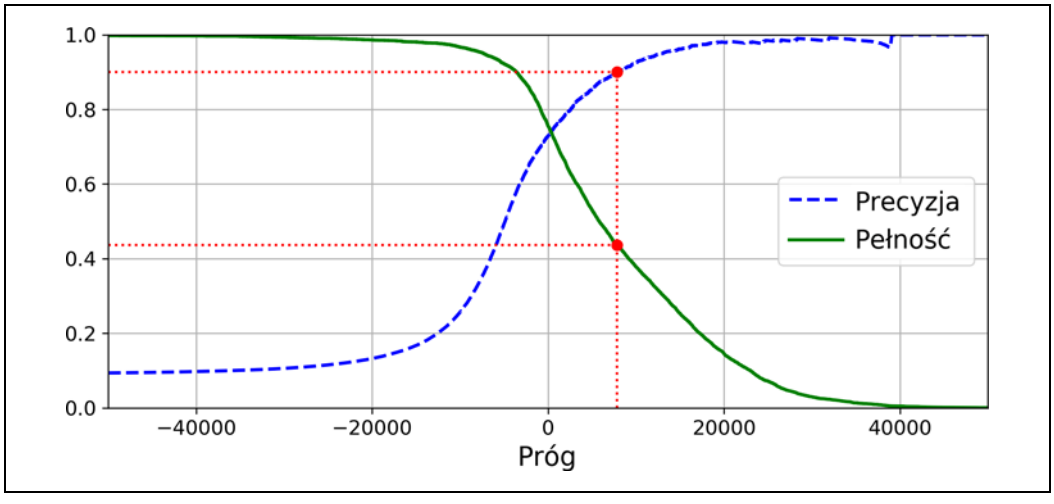
```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
```

Na koniec za pomocą modułu Matplotlib wygeneruj wykres precyzji i pełności w funkcji progu decyzyjnego (rysunek 3.4):

```
def plot_precision_recall_vs_threshold(precisions, recalls, thresholds):
    plt.plot(thresholds, precisions[:-1], "b--", label="Precyzja")
    plt.plot(thresholds, recalls[:-1], "g-", label="Pełność")
    […] # Zaznaczenie progu, dodanie legendy, etykiet osi i siatki

plot_precision_recall_vs_threshold(precisions, recalls, thresholds)
plt.show()
```



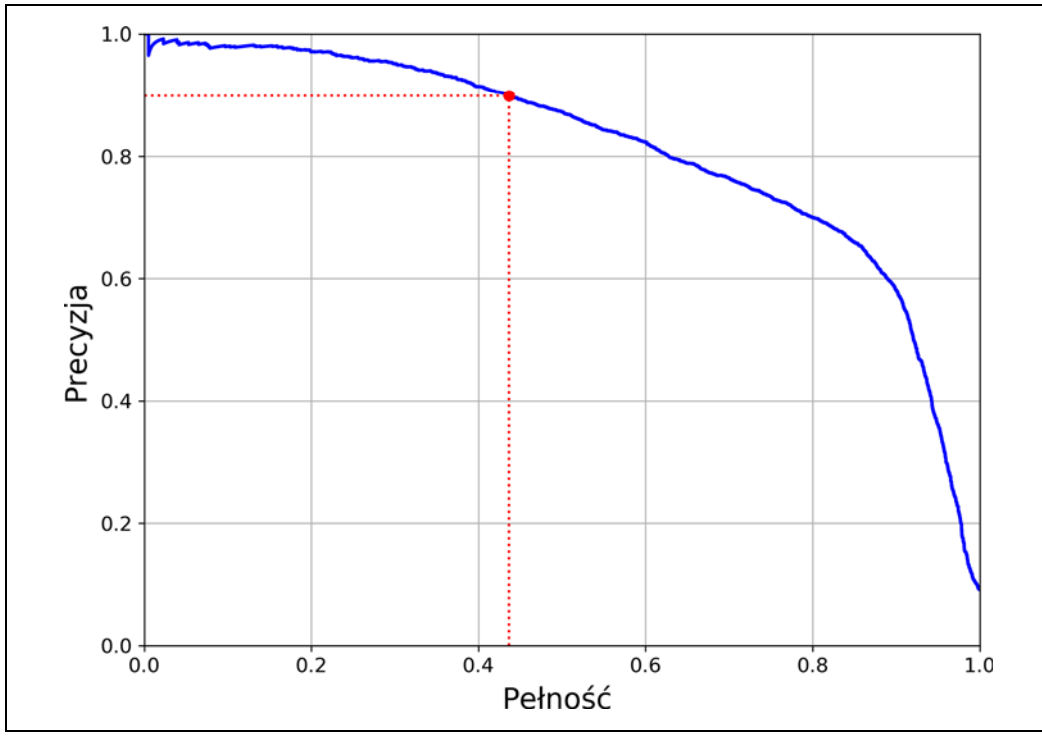
Rysunek 3.4. *Wykres precyzji i pełności w funkcji progu decyzyjnego*



Pewnie zastanawiasz się, dlaczego na rysunku 3.4 krzywa precyzji nie jest tak równa, jak krzywa pełności. Wynika to z faktu, że precyzja może czasami maleć wraz z podnoszeniem wartości progowej (chociaż zasadniczo jej wartość również wzrasta). Aby zrozumieć ten mechanizm, przyjrzyj się ponownie rysunkowi 3.3; zobacz, co się będzie działo, gdy zaczniesz od środkowego progu i będziesz przesuwać się w prawo po jednej pozycji naraz: wartość precyzji zmienia się z 4/5 (80%) na 3/4 (75%). Z drugiej strony, wraz ze wzrostem wartości progowej pełność może wyłącznie maleć, dlatego jej krzywa jest taka gładka.

Innym sposobem wyznaczenia dobrego kompromisu jest narysowanie wykresu precyzji bezpośrednio w funkcji pełności, co zostało zaprezentowane na rysunku 3.5 (wyznaczyliśmy tu ten sam próg co poprzednio).

Jak widać, krzywa precyzji zaczyna stromo opadać dopiero przy wartości mniej więcej 80% pełności. Prawdopodobnie będziemy chcieli wybrać kompromis pomiędzy precyzją a pełnością wyznaczony tuż przed początkiem opadania krzywej — mniej więcej przy 60% pełności. Oczywiście wybór odpowiedniego progu zależy od danego projektu.



Rysunek 3.5. *Wykres precyzji w funkcji pełności*

Załóżmy, że zależy Ci na precyzji rzędu 90%. Przyglądamy się pierwszemu wykresowi (w razie potrzeby nieznacznie go przybliżamy) i dowiadujemy się, że powinniśmy użyć progu o wartości ok. 8000. Dla zwiększenia dokładności możesz poszukać najmniejszego progu zapewniającego precyzję rzędu do najmniej 90% (funkcja `np.argmax()` wyznaczy pierwszy indeks wartości maksymalnej, co w tym przypadku oznacza pierwszą wartość `true`):

```
threshold_90_precision = thresholds[np.argmax(precisions >= 0.90)] # ~7816
```

Aby wyliczyć prognozy (na razie na zbiorze uczącym), nie będziemy wywoływać metody `predict()` klasyfikatora, lecz skorzystamy z następującego kodu:

```
y_train_pred_90 = (y_scores > threshold_90_precision)
```

Sprawdźmy teraz precyzję i pełność tych prognoz:

```
>>> precision_score(y_train_5, y_train_pred_90)
0.9000380083618396
>>> recall_score(y_train_5, y_train_pred_90)
0.4368197749492714
```

Świetnie! Otrzymaliśmy klasyfikator cechujący się 90-procentową precyzją! Jak widać, nie jest trudno uzyskać klasyfikator mający praktycznie dowolną wybraną przez nas precyzję: wystarczy wyznaczyć odpowiednio wysoki próg i już. Chwilunia, nie tak szybko. Precyzyjny klasyfikator nie jest zbyt przydatny, jeśli ma za małą pełność!



Jeśli ktoś powie: „Uzyskajmy 99% precyzji”, zapytaj: „A przy jakiej wartości pełności?”.

Wykres krzywej ROC

Wykres krzywej **charakterystyki roboczej odbiornika** (ang. *receiver operating characteristic* — ROC) stanowi kolejne popularne narzędzie używane wraz z klasyfikatorami binarnymi. Przypomina on w znacznym stopniu wykres precyzji w funkcji pełności, jednak w tym przypadku rysujemy **odsetek prawdziwie pozytywnych** (inna nazwa pełności; w skrócie OPP) w funkcji **odsetka fałszywie pozytywnych** (OFP) — jest to odsetek negatywnych przykładów, które zostały nieprawidłowo sklasyfikowane jako pozytywne. Otrzymujemy tę wartość, **odejmując odsetek prawdziwie negatywnych** (OPN) od liczby 1; jak można się domyślić, jest to odsetek negatywnych próbek, które zostały prawidłowo sklasyfikowane jako negatywne. Wartość ta bywa również nazywana **specyficznością**. Dlatego krzywa ROC składa się z **czułości** (czyli pełności) w funkcji **1 – specyficzność**.

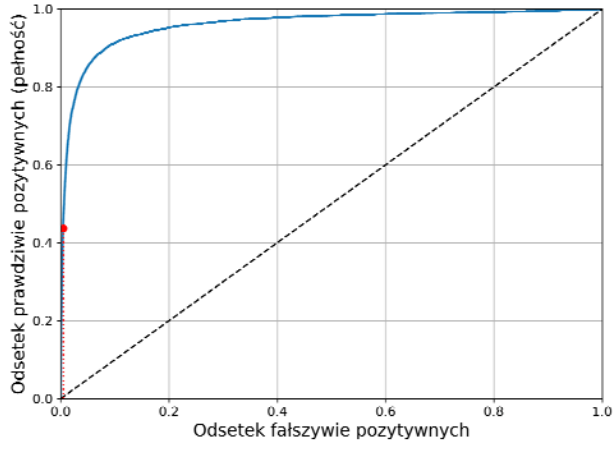
Aby narysować wykres krzywej ROC, wykorzystujemy funkcję roc_curve() do obliczenia przykładów OPP i OFP dla różnych wartości progowych:

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

Teraz przy użyciu modułu Matplotlib możemy narysować wykres OPP w funkcji OFP. Poniższy kod generuje wykres widoczny na rysunku 3.6:

```
def plot_roc_curve(fpr, tpr, label=None):
   plt.plot(fpr, tpr, linewidth=2, label=label)
   plt.plot([0, 1], [0, 1], 'k--') # Przerywana przekątna wykresu
   […] # Dodanie etykiety osi i siatki
plot_roc_curve(fpr, tpr)
plt.show()
```



Rysunek 3.6. Wykres krzywej ROC

Także w tym przypadku mamy do czynienia z kompromisem: im wyższa wartość pełności (OPP), tym więcej fałszywie pozytywnych (OFP) uzyskuje klasyfikator. Linia przerywana symbolizuje krzywą ROC całkowicie losowego klasyfikatora; dobry klasyfikator stara się nie zbliżać do tej prostej (czyli dąży do lewej górnej części wykresu).

Jednym ze sposobów porównywania klasyfikatorów jest pomiar ich **obszaru pod krzywą** (ang. *area under the curve* — AUC). Obszar AUC w przypadku doskonałego klasyfikatora byłby równy 1, natomiast w całkowicie losowym klasyfikatorze ma on wartość 0,5. Moduł Scikit-Learn zawiera funkcję służącą do wyliczania obszaru AUC na wykresie ROC:

```
>>> from sklearn.metrics import roc_auc_score
>>> roc_auc_score(y_train_5, y_scores)
0.9611778893101814
```



Skoro wykres krzywej ROC tak bardzo przypomina wykres krzywej precyzji w funkcji pełności (ang. *precision/recall curve* — PR), skąd mamy wiedzieć, którego z nich użyć? Zgodnie z niepisaną zasadą powinniśmy korzystać z wykresu PR wtedy, gdy klasa pozytywna występuje rzadko lub gdy bardziej Ci zależy na odsetku fałszywie pozytywnych niż fałszywie negatywnych. Z kolei wykres ROC stosujemy w pozostałych sytuacjach. Przykładowo, jeśli spojrzymy na powyższy wykres krzywej ROC (oraz na wynik obszaru pod tą krzywą), moglibyśmy uznać, że odpowiedzialny za nią klasyfikator jest naprawdę dobry. Wynika to jednak głównie z faktu, że w zestawie danych istnieje niewiele wartości pozytywnych (piątek) w stosunku do negatywnych (niepiątek). Z drugiej strony, wykres krzywej PR wyraźnie pokazuje, że klasyfikator ten można jeszcze usprawnić (krzywa powinna jeszcze bardziej dążyć do lewego górnego rogu wykresu).

Wyuczmy teraz klasyfikator `RandomForestClassifier` i porównajmy jego krzywą ROC oraz obszar AUC z wynikami uzyskanymi dla klasyfikatora `SGDClassifier`. Najpierw musimy obliczyć wyniki dla każdego przykładu ze zbioru uczącego. Jednak z powodu mechanizmu działania (zob. rozdział 7.) klasa `RandomForestClassifier` nie zawiera metody `decision_function()`. Znajdziemy tu jednak metodę `predict_proba()`. Klasyfikatory modułu Scikit-Learn przeważnie zawierają którąś z tych dwóch metod albo obydwie. Metoda `predict_proba()` zwraca tablicę, w której każdy wiersz reprezentuje próbkę, a kolumna — klasę; wyliczone są w niej prawdopodobieństwa przynależności danego przykładu do określonej klasy (np. 70% szans, że dana próbka jest piątką):

```
from sklearn.ensemble import RandomForestClassifier

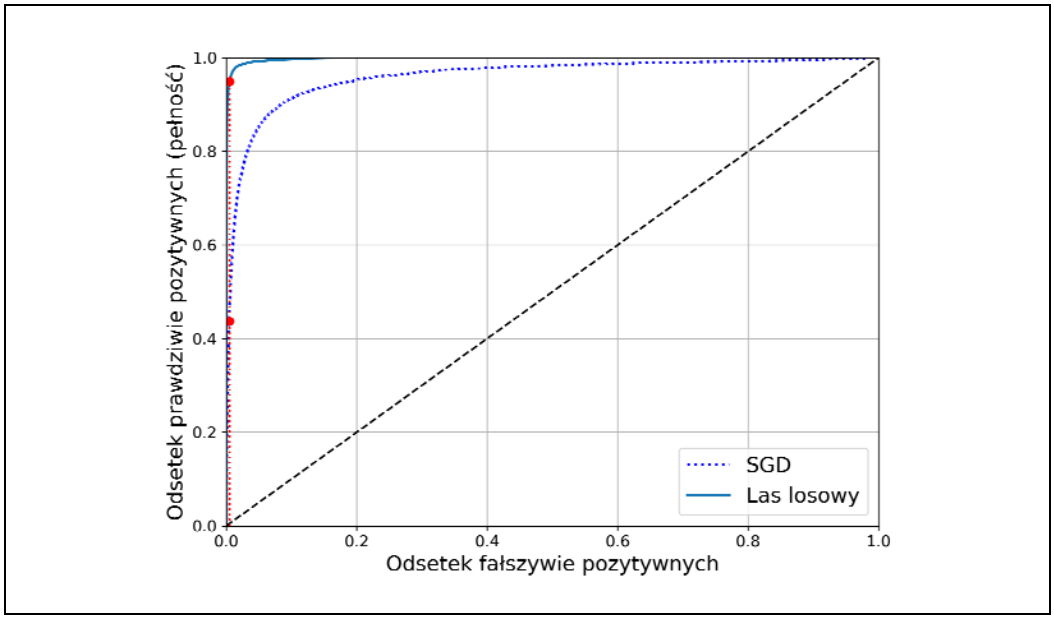
forest_clf = RandomForestClassifier(random_state=42)
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,
                                    method="predict_proba")
```

Funkcja `roc_curve()` oczekuje etykiet i wyników, ale zamiast tych drugich możemy skorzystać z prawdopodobieństwa przynależności do klasy. Użyjmy prawdopodobieństwa przynależności do klasy pozytywnej jako wyniku:

```
y_scores_forest = y_probas_forest[:, 1] # Wynik = prawd. przynależności do klasy pozytywnej
fpr_forest, tpr_forest, thresholds_forest = roc_curve(y_train_5,y_scores_forest)
```

Jesteśmy teraz w stanie narysować wykres krzywej ROC. Warto również wyrysować wcześniejszą krzywą ROC, aby móc porównać obydwa modele (rysunek 3.7):

```
plt.plot(fpr, tpr, "b:", label="SGD")
plot_roc_curve(fpr_forest, tpr_forest, "Las losowy")
plt.legend(loc="lower right")
plt.show()
```



Rysunek 3.7. *Porównanie krzywych ROC: klasyfikator lasu losowego jest lepszy od klasyfikatora SGD, ponieważ jego krzywa ROC znajduje się znacznie bliżej górnego lewego rogu, a jego obszar AUC jest także korzystniejszy*

Jak widać na rysunku 3.7, krzywa ROC klasyfikatora `RandomForestClassifier` wygląda znacznie lepiej od krzywej klasyfikatora `SGDClassifier`: dociera ona znacznie bliżej lewego górnego rogu wykresu. W związku z tym jego wynik obszaru AUC jest również o wiele lepszy:

```
>>> roc_auc_score(y_train_5, y_scores_forest)
0.9983436731328145
```

Spróbuj zmierzyć również wartości precyzji i pełności: powinnaś/powinieneś uzyskać wyniki rzędu, odpowiednio, 99% i 86.6%. Całkiem nieźle!

Potrafisz już trenować klasyfikatory binarne, wybierać odpowiedni wskaźnik dla danego zadania, oceniać wydajność klasyfikatorów za pomocą sprawdzianu krzyżowego, dobierać do swoich potrzeb odpowiedni kompromis pomiędzy precyzją a pełnością oraz porównywać modele przy użyciu krzywych ROC i wyników AUC. Spróbujmy teraz wykrywać jakieś inne cyfry oprócz piątek.

Klasyfikacja wieloklasowa

Klasyfikatory binarne służą do rozpoznawania dwóch klas, natomiast **klasyfikatory wieloklasowe** (zwane również **klasyfikatorami wielomianowymi**) służą do rozróżniania większej ich liczby.

Niektóre algorytmy (takie jak klasyfikatory SGD, klasyfikatory losowego lasu lub naiwne klasyfikatory bayesowskie) są w stanie natywnie zajmować się wieloma klasami naraz. Inne (takie jak maszyny wektorów nośnych czy klasyfikatory regresji logistycznej) mają charakter typowo binarny. Istnieje jednak wiele strategii, dzięki którym możemy przeprowadzać klasyfikację wieloklasową przy użyciu wielu klasyfikatorów binarnych.

Jeden ze sposobów stworzenia systemu klasyfikującego 10 klas obrazów cyfr (od 0 do 9) polega na wyuczeniu 10 klasyfikatorów binarnych, po jednym na każdą cyfrę (wykrywacz zer, jedynek, dwójek itd.). Następnie na etapie klasyfikacji obrazu uzyskujemy wynik z każdego klasyfikatora dla danej cyfry i wybieramy klasę, która uzyskała najwyższy wynik. Jest to tzw. strategia **jeden przeciw reszcie** (ang. *one-versus-rest* — OvR), znana również jako **jeden przeciw wszystkim** (ang. *one-versus-all* — OvA).

Jeszcze innym rozwiązaniem jest wyuczenie klasyfikatora binarnego dla każdej pary cyfr: jeden rozpoznaje zera i jedynki, drugi — zera i dwójki, jeszcze inny — jedynki i dwójki itd. Jest to strategia **jeden przeciw jednemu** (ang. *one-versus-one* — OvO). Jeśli masz do czynienia z N klasami, musisz wytrenować N×(N–1):2 klasyfikatorów. W przypadku zestawu danych MNIST oznacza to wyuczenie 45 klasyfikatorów binarnych! Aby zaklasyfikować dany obraz, musielibyśmy przepuścić go przez wszystkie 45 klasyfikatorów i sprawdzić, która klasa wygrywa większość „pojedynków". Główną zaletą strategii OvO jest fakt, że każdy klasyfikator musi zostać wytrenowany jedynie wobec części zbioru uczącego składającego się z obydwu porównywanych klas.

Pewne algorytmy (jak choćby maszyny wektorów nośnych) nie skalują się zbyt dobrze do rozmiarów zbioru uczącego. W ich przypadku preferowane jest stosowanie strategii OvO, ponieważ uczenie wielu klasyfikatorów wobec niewielkich zbiorów danych przebiega szybciej niż trenowanie kilku klasyfikatorów wobec dużych zbiorów przykładów. Jednakże w przypadku większości klasyfikatorów binarnych zalecane jest korzystanie ze strategii OvR.

Moduł Scikit-Learn wykrywa sytuacje, gdy próbujemy używać algorytmu klasyfikacji binarnej w procesie klasyfikacji wieloklasowej, i automatycznie przechodzi na strategię OvR lub OvO, w zależności od algorytmu. Sprawdźmy to na klasyfikatorze maszyny wektorów nośnych (zob. rozdział 5.), za pomocą klasy `sklearn.svm.SVC`:

```
>>> from sklearn.svm import SVC
>>> svm_clf = SVC()
>>> svm_clf.fit(X_train, y_train) # y_train, a nie y_train_5
>>> svm_clf.predict([some_digit])
array([5], dtype=uint8)
```

To było proste! Powyższy fragment kodu wyucza klasyfikator `SVC` wobec zbioru uczącego za pomocą pierwotnych klas docelowych w zakresie od 0 do 9 (`y_train`), zatem nie korzystamy tu z klas docelowych 5 przeciw reszcie (`y_train_5`). Następnie zostaje wyliczona prognoza (tym razem prawidłowa). Zgodnie z tą strategią moduł Scikit-Learn skorzystał w rzeczywistości ze strategii OvO: wytrenował 45 klasyfikatorów binarnych, uzyskał wyniki decyzyjne dla analizowanego obrazu i wybrał klasę, która wygrała większość „pojedynków".

Jeżeli wywołasz metodę `decision_function()`, zauważysz, że zostaje zwrócony nie jeden wynik na obraz, ale aż 10 wyników, po jednym na każdą klasę:

```
>>> some_digit_scores = svm_clf.decision_function([some_digit])
>>> some_digit_scores
array([[2.92492871, 7.02307409, 3.93648529, 0.90117363, 5.96945908,
9.5, 1.90718593, 8.02755089, -0.13202708, 4.94216947]])
```

Istotnie, najwyższy wynik dotyczy klasy 5:

```
>>> np.argmax(some_digit_scores)
5
>>> svm_clf.classes_
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], dtype=uint8)
>>> svm_clf.classes_[5]
5
```



Klasyfikator po wytrenowaniu przechowuje listę klas docelowych we własnym atrybucie `classes_`, w uporządkowanym szeregu kolejności. W naszym przypadku indeks każdej klasy w tablicy `classes_` dogodnie dla nas jest dopasowany do każdej klasy (np. okazuje się, że klasa o indeksie 5 symbolizuje klasę piątek), zazwyczaj jednak nie powinniśmy liczyć na takie szczęście.

Jeśli chcesz zmusić moduł Scikit-Learn do stosowania strategii OvO lub OvR, możesz skorzystać z klas `OneVsOneClassifier` lub `OneVsRestClassifier`. Wystarczy stworzyć ich wystąpienie i przekazać konstruktorowi klasyfikator (nie musi być on nawet binarny). Na przykład poniższy kod stworzy klasyfikator wieloklasowy (na podstawie klasyfikatora `SVC`) przy użyciu strategii OvR:

```
>>> from sklearn.multiclass import OneVsRestClassifier
>>> ovr_clf = OneVsRestClassifier(SVC())
>>> ovr_clf.fit(X_train, y_train)
>>> ovr_clf.predict([some_digit])
array([5], dtype=uint8)
>>> len(ovr_clf.estimators_)
10
```

Równie łatwe jest uczenie klasyfikatora `SGDClassifier` (lub `RandomForestClassifier`):

```
>>> sgd_clf.fit(X_train, y_train)
>>> sgd_clf.predict([some_digit])
array([5], dtype=uint8)
```

Tym razem nie zostały wdrożone strategie OvO ani OvR, ponieważ klasyfikatory SGD mogą bezpośrednio klasyfikować przykłady w sposób wieloklasowy. Metoda `decision_function()` zwraca teraz po jednej wartości na każdą klasę. Spójrzmy na wynik wyznaczony każdej klasie przez klasyfikator SGD:

```
>>> sgd_clf.decision_function([some_digit])
array([[-15955.22628, -38080.96296, -13326.66695, 573.52692, -17680.68466,
2412.53175, -25526.86498, -12290.15705, -7946.05205, -10631.35889]])
```

Jak widać, klasyfikator wykazuje dość duży stopień pewności w swoich przewidywaniach: niemal wszystkie wyniki mają dużą wartość ujemną, podczas gdy klasa 5 uzyskała wynik 2412,5. Model wykazuje drobne wątpliwości w przypadku klasy 3, której wynik to 573,5. Teraz oczywiście chcemy ocenić skuteczność tych klasyfikatorów. Jak zwykle użyjemy w tym celu sprawdzianu krzyżowego. Sprawdźmy dokładność klasyfikatora `SGDClassifier` za pomocą funkcji `cross_val_score()`:

```
>>> cross_val_score(sgd_clf, X_train, y_train, cv=3, scoring="accuracy")
array([0.8489802, 0.87129356, 0.86988048])
```

Uzyskujemy wynik 84% wobec wszystkich podzbiorów testowych. Gdybyśmy korzystali z losowego klasyfikatora, otrzymalibyśmy wynik 10% dokładności, zatem nie jest źle, ale mogłoby być znacznie lepiej. Po zwyczajnym przeskalowaniu danych wejściowych (zostało to omówione w rozdziale 2.) osiągnęlibyśmy dokładność powyżej 89%:

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler()
>>> X_train_scaled = scaler.fit_transform(X_train.astype(np.float64))
>>> cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3, scoring="accuracy")
array([0.89707059, 0.8960948 , 0.90693604])
```

Analiza błędów

Gdybyśmy mieli do czynienia z rzeczywistym projektem, przeprowadzalibyśmy teraz czynności wymienione w dodatku B. Sprawdzalibyśmy możliwości przygotowywania danych, testowalibyśmy różne modele, stworzylibyśmy krótką listę najlepszych modeli i dostroilibyśmy ich hiperparametry za pomocą klasy `GridSearchCV`, a do tego próbowalibyśmy w jak największym stopniu zautomatyzować cały proces. Obecnie założymy, że znaleźliśmy obiecujący model, i szukamy sposobów, aby go jeszcze usprawnić. Jedną z możliwości jest analiza rodzajów błędów popełnianych przez ten model.

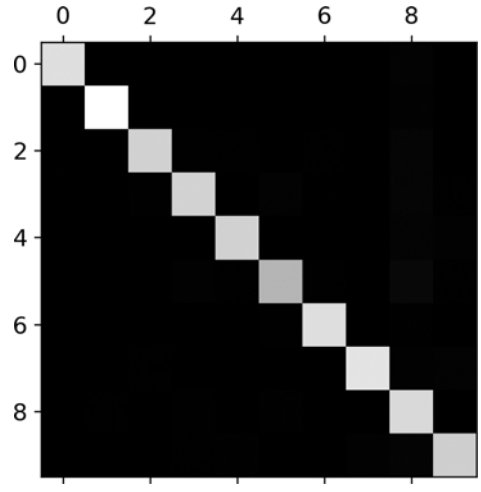
Po pierwsze, przyjrzyjmy się macierzy pomyłek. Podobnie jak robiliśmy to wcześniej, musimy wyliczyć prognozy za pomocą funkcji `cross_val_predict()`, a następnie wywołać funkcję `confusion_matrix()`:

```
>>> y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
>>> conf_mx = confusion_matrix(y_train, y_train_pred)
>>> conf_mx
array([[5578,    0,   22,    7,    8,   45,   35,    5,  222,    1],
       [   0, 6410,   35,   26,    4,   44,    4,    8,  198,   13],
       [  28,   27, 5232,  100,   74,   27,   68,   37,  354,   11],
       [  23,   18,  115, 5254,    2,  209,   26,   38,  373,   73],
       [  11,   14,   45,   12, 5219,   11,   33,   26,  299,  172],
       [  26,   16,   31,  173,   54, 4484,   76,   14,  482,   65],
       [  31,   17,   45,    2,   42,   98, 5556,    3,  123,    1],
       [  20,   10,   53,   27,   50,   13,    3, 5696,  173,  220],
       [  17,   64,   47,   91,    3,  125,   24,   11, 5421,   48],
       [  24,   18,   29,   67,  116,   39,    1,  174,  329, 5152]])
```

Całkiem sporo tych liczb. Często wygodniej jest spojrzeć na graficzną reprezentację macierzy pomyłek za pomocą funkcji `matshow()`:

```
plt.matshow(conf_mx, cmap=plt.cm.gray)
plt.show()
```

Macierz ta wygląda całkiem nieźle, gdyż większość obrazów znajduje się na głównej przekątnej, co oznacza, że zostały prawidłowo sklasyfikowane. Piątki wyglądają nieco ciemniej od pozostałych cyfr, co może oznaczać, że w zbiorze danych znajduje się mniej obrazów piątek lub że klasyfikator nie sprawuje się tak dobrze wobec piątek, jak w przypadku pozostałych cyfr. W rzeczywistości możemy zweryfikować obydwa stwierdzenia.

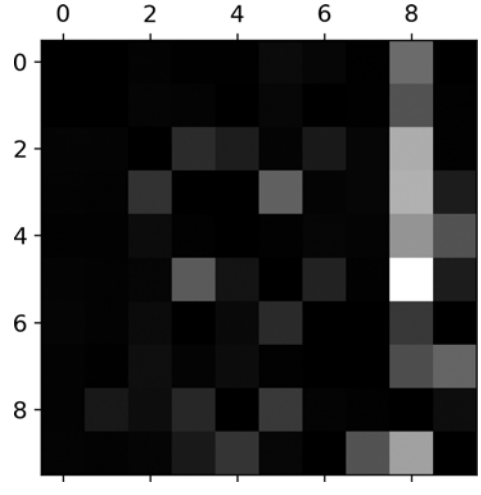


Skoncentrujmy się na narysowaniu wykresu błędów. Najpierw musimy podzielić każdą wartość w macierzy pomyłek przez liczbę obrazów należących do danej klasy, dzięki czemu będziemy w stanie porównać poziomy błędu zamiast bezwzględnej liczby błędów (gdybyśmy przy niej pozostali, klasy zawierające liczne przykłady wypadałyby nazbyt niekorzystnie):

```
row_sums = conf_mx.sum(axis=1, keepdims=True)
norm_conf_mx = conf_mx / row_sums
```

Wypełnijmy główną przekątną zerami po to, aby pozostały same błędy, i znowu wyświetlmy macierz pomyłek w postaci graficznej:

```
np.fill_diagonal(norm_conf_mx, 0)
plt.matshow(norm_conf_mx, cmap=plt.cm.gray)
plt.show()
```



Wyraźnie widzimy rodzaje błędów popełnianych przez klasyfikator. Przypominam, że rzędy reprezentują rzeczywiste klasy, kolumny zaś — przewidywane klasy. Kolumna reprezentująca klasę 8 jest

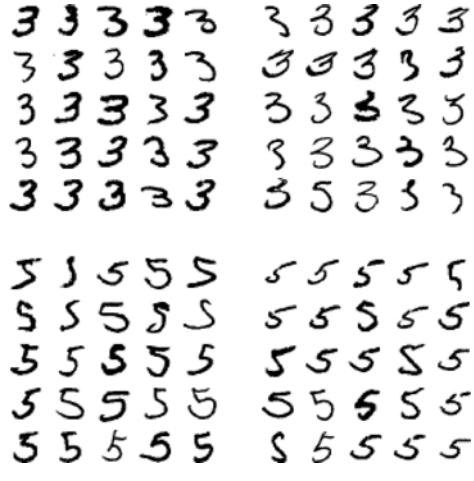
dość jasna, co oznacza, że wiele obrazów zostało nieprawidłowo sklasyfikowanych jako ta właśnie cyfra. Jednak rząd klasy 8 nie wygląda tak źle, co oznacza, że zasadniczo większość ósemek jest prawidłowo klasyfikowanych. Jak widać, macierz pomyłek nie jest zbyt symetryczna. Możemy również dostrzec, że trójki i piątki są często mylone (w obydwie strony).

Analiza macierzy pomyłek często dostarcza wskazówek pozwalających usprawnić klasyfikator. Patrząc na nią, możemy stwierdzić, że należy popracować nad zmniejszeniem liczby fałszywych ósemek. Możesz na przykład spróbować zdobyć więcej obrazów cyfr przypominających ósemki (które jednak nimi nie są), dzięki czemu klasyfikator nauczy się je odróżniać od rzeczywistych ósemek. Ewentualnie moglibyśmy spróbować stworzyć nowe cechy pomagające klasyfikatorowi — przykładowo, napisać algorytm zliczający liczbę zamkniętych pętli na obrazie (np. cyfra 8 ma dwie pętle, szóstka ma jedną, a piątka żadnej). Jeszcze innym rozwiązaniem jest wstępne przetwarzanie obrazów (np. za pomocą modułów Scikit-Image, Pillow lub OpenCV) w taki sposób, aby uwypuklić pewne wzorce, takie jak wspomniane już zamknięte pętle.

Dzięki analizowaniu poszczególnych błędów możemy dowiedzieć się również, jak właściwie działa nasz klasyfikator i jaka jest przyczyna pomyłek, proces ten jest jednak bardziej skomplikowany i czasochłonny. Narysujmy, na przykład, wykres przykładowych trójek i piątek (funkcja `plot_digits()` wykorzystuje po prostu funkcję `imshow()` modułu Matplotlib; szczegóły znajdziesz w notatniku Jupyter dołączonym do tego rozdziału):

```
cl_a, cl_b = 3, 5
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]

plt.figure(figsize=(8,8))
plt.subplot(221); plot_digits(X_aa[:25], images_per_row=5)
plt.subplot(222); plot_digits(X_ab[:25], images_per_row=5)
plt.subplot(223); plot_digits(X_ba[:25], images_per_row=5)
plt.subplot(224); plot_digits(X_bb[:25], images_per_row=5)
plt.show()
```



Dwa bloki o rozmiarze 5×5 widoczne po lewej stronie przedstawiają cyfry sklasyfikowane jako trójki, natomiast po prawej stronie widzimy prognozowane piątki. Niektóre nieprawidłowo rozpoznane cyfry (np. w lewym dolnym i prawym górnym bloku) zostały tak fatalnie zapisane, że nawet człowiek miałby problem z ich klasyfikacją (np. piątka znajdująca się na przecięciu pierwszego rzędu i drugiej kolumny wygląda jak wyjątkowo niechlujnie zapisana trójka). Jednak większość nieprawidłowo sklasyfikowanych obrazów to z perspektywy ludzkiego mózgu wyraźne błędy i ciężko nam zrozumieć, dlaczego klasyfikator popełnił takie gafy[3]. Wynika to z faktu, że skorzystaliśmy z klasy `SGDClassifier`, która jest w istocie prostym modelem liniowym. Przydziela on jedynie wagi przynależności do klasy dla każdego piksela, a w trakcie analizy nowej próbki po prostu sumuje ważone poziomy szarości pikseli, aby uzyskać wynik dla każdej klasy. Skoro więc trójki i piątki różnią się zaledwie kilkoma pikselami, klasyfikator może łatwo się mylić w ich rozróżnianiu.

Główną różnicą pomiędzy trójkami i piątkami jest położenie małej linii łączącej górną kreskę z dolnym łukiem. Jeśli narysujemy cyfrę trzy z tym połączeniem przesuniętym nieznacznie w lewą stronę, klasyfikator może ją uznać za piątkę, i odwrotnie. Innymi słowy, klasyfikator ten jest czuły na przesunięcia i obroty. Z tego wynika, że w celu ułatwienia rozróżniania trójek od piątek należałoby wstępnie przetworzyć obrazy tak, aby zostały wyśrodkowane i jak najmniej przechylone. Prawdopodobnie rozwiązanie to zmniejszyłoby występowanie również innych błędów.

Klasyfikacja wieloetykietowa

Do tej pory każdy przykład był przydzielany wyłącznie do jednej klasy. W niektórych sytuacjach chcemy, aby klasyfikator wyznaczał wiele klas dla jednego wystąpienia. Weźmy pod uwagę klasyfikator rozpoznawania twarzy: co powinien zrobić, jeśli rozpozna kilka osób na jednym zdjęciu? Powinien przydzielić po jednej etykiecie na każdą rozpoznaną osobę. Załóżmy, że model taki został wyuczony do rozpoznawania trzech osób: Alicji, Władka i Karola. Po zaprezentowaniu zdjęć Alicji i Karola powinien zostać wygenerowany wynik [1, 0, 1] (czyli „Alicja tak, Władek nie, Karol tak"). Tego typu system klasyfikujący zdolny do wyznaczania wielu binarnych znaczników nosi nazwę systemu **klasyfikacji wieloetykietowej** (ang. *multilabel classification*).

Nie będziemy jeszcze zajmować się zagadnieniem rozpoznawania twarzy, ale przyjrzyjmy się prostszemu przykładowi, wyłącznie w celach poglądowych:

```
from sklearn.neighbors import KNeighborsClassifier

y_train_large = (y_train >= 7)
y_train_odd = (y_train % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

Powyższy kod generuje tablicę `y_multilabel` zawierającą dwie docelowe etykiety dla każdego obrazu cyfry: pierwsza z nich określa, czy mamy do czynienia z cyfrą o dużej wartości (7, 8 i 9), natomiast

[3] Nie zapominaj jednak, że nasz mózg stanowi doskonały system rozpoznawania wzorców, a układ wzrokowy przeprowadza mnóstwo wstępnych modyfikacji obrazu, zanim trafi on do naszej świadomości, dlatego cechująca nas łatwość klasyfikowania obrazów wcale nie oznacza, że jest to prosty proces.

druga mówi nam, czy dana cyfra jest nieparzysta. Następnie zostaje utworzone wystąpienie klasyfikatora `KNeighborsClassifier` (obsługuje on klasyfikację wieloetykietową, czego nie można powiedzieć o wszystkich klasyfikatorach), po czym uczymy go za pomocą tablicy zawierającej wiele docelowych elementów. Teraz możemy wyliczyć prognozy, w wyniku czego otrzymujemy dwie etykiety:

```
>>> knn_clf.predict([some_digit])
array([[False, True]])
```

Zgadza się! Cyfra 5 rzeczywiście nie ma dużej wartości (`False`) i jest nieparzysta (`True`).

Istnieje wiele sposobów oceniania wydajności klasyfikatora wieloetykietowego, a dobór wskaźnika zależy w rzeczywistości od natury projektu. Jedną z metod jest obliczenie wyniku F_1 dla poszczególnych etykiet (lub użycie dowolnej innej omówionego wcześniej wskaźnika klasyfikatora binarnego), a następnie po prostu wyliczenie średniej. Poniższy kod wylicza uśredniony wynik F_1 dla wszystkich etykiet:

```
>>> y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
>>> f1_score(y_multilabel, y_train_knn_pred, average="macro")
0.976410265560605
```

Zakładamy tu jednak, że wszystkie etykiety mają taką samą wagę, co nie zawsze jest prawdą. Jest to istotne zwłaszcza w sytuacji, gdybyśmy mieli znacznie więcej zdjęć Alicji niż Władka lub Karola — należałoby wtedy wyznaczyć większą wagę dla wyników zdjęć Alicji. Prostym rozwiązaniem jest nadanie każdej etykiecie wagi równej jej **współczynnikowi wsparcia** (ang. *support*; np. liczbie próbek zawierających etykietę docelową). W tym celu wystarczy w powyższym fragmencie kodu wstawić wyrażenie `average="weighted"`[4].

Klasyfikacja wielowyjściowa

Ostatnim rodzajem zadania klasyfikacyjnego, jakie omówimy, jest **klasyfikacja wielowyjściowo-wieloklasowa** (ang. *multioutput-multiclass classification*), zwana również **klasyfikacją wielowyjściową** (ang. *multioutput classification*). Jest to po prostu uogólnienie klasyfikacji wieloetykietowej, w którym każda etykieta może być wieloklasowa (tj. może mieć ponad dwie możliwe wartości).

Aby to zilustrować, stwórzmy system usuwający szumy z obrazów cyfr. Danymi wejściowymi będą zaszumione obrazy cyfr, na wyjściu natomiast uzyskamy (taką mamy nadzieję) czysty obraz cyfry w postaci tablicy zawierającej poziomy szarości poszczególnych pikseli, zupełnie jak w przypadku zestawu danych MNIST. Zwróć uwagę, że wynik klasyfikatora jest wieloetykietowy (na jeden piksel przypada jedna etykieta), a każda z etykiet może mieć różne wartości (poziomy szarości piksela mieszczą się w zakresie od 0 do 255). Jak zatem widać, jest to doskonały przykład systemu klasyfikacji wielowyjściowej.

[4] Moduł Scikit-Learn zawiera kilka innych opcji uśredniania oraz wskaźników klasyfikatora wieloetykietowego; szczegóły znajdziesz w dokumentacji.



Granica pomiędzy klasyfikacją a regresją bywa czasami bardzo zamazana, tak jak w omawianym przypadku. Zapewne przewidywanie poziomu szarości pikseli bardziej przypomina regresję niż klasyfikację. Do tego systemy wielowyjściowe nie ograniczają się wyłącznie do zadań klasyfikacji; możesz nawet stworzyć system, którego rezultatami jest wiele etykiet na każdy przykład, w tym zarówno etykiety klas, jak i etykiety wartości.

Zacznijmy od utworzenia zbiorów uczących i testowych — za pomocą funkcji `randint()` dołączymy zaszumienie do poziomów szarości pikseli w obrazach zbioru danych MNIST. Obrazami docelowymi będą pierwotne próbki:

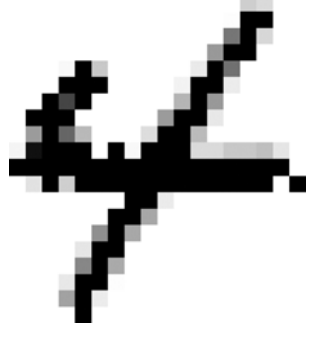
```
noise = np.random.randint(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise
noise = np.random.randint(0, 100, (len(X_test), 784))
X_test_mod = X_test + noise
y_train_mod = X_train
y_test_mod = X_test
```

Zobaczmy, jak wygląda przykładowy obraz ze zbioru testowego (tak, zaglądamy do danych testowych, dlatego wskazany jest srogi mars na Twoim czole):



Po lewej stronie widzimy zaszumiony obraz wejściowy, a po prawej — czysty obraz docelowy. Wyuczmy teraz nasz klasyfikator i oczyśćmy za jego pomocą obraz:

```
knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[some_index]])
plot_digit(clean_digit)
```



Wygląda całkiem podobnie do obrazu docelowego! Na tym zakończymy naszą wycieczkę po krainie klasyfikacji. Umiesz już dobierać odpowiednie wskaźniki do zadań klasyfikacji, wyznaczać optymalny kompromis pomiędzy precyzją a pełnością, porównywać klasyfikatory, a także — bardziej ogólnie — tworzyć dobre systemy klasyfikujące dopasowane do różnorodnych zadań.

Ćwiczenia

1. Spróbuj stworzyć klasyfikator zbioru danych MNIST osiągający ponad 97% dokładności dla zbioru testowego. Podpowiedź: całkiem nieźle sprawdza się klasyfikator `KNeighborsClassifier`; musisz tylko dobrać odpowiednie wartości hiperparametrów (zastosuj metodę przeszukiwania siatki wobec hiperparametrów `weights` i `n_neighbors`).
2. Napisz funkcję przesuwającą obraz z zestawu danych MNIST o jeden piksel w dowolnym kierunku (w lewo, prawo, do góry lub w dół)[5]. Następnie dla każdego obrazu w zbiorze uczącym stwórz cztery przesunięte kopie (po jednej na każdy kierunek przesunięcia) i dołącz je do zbioru uczącego. Na koniec wytrenuj swój najlepszy model wobec takiego rozszerzonego zestawu danych i zmierz jego dokładność przy użyciu zbioru testowego. Twój model powinien teraz sprawować się jeszcze lepiej! Taka technika sztucznego powiększania zestawu uczącego bywa nazywana **dogenerowaniem danych** (ang. *data augmentation*) lub **rozszerzeniem zbioru uczącego** (ang. *training set expansion*).
3. Pobaw się zbiorem danych *Titanic*. Warto rozpocząć od serwisu Kaggle (*https://www.kaggle.com/c/titanic*).
4. Trudniejsze zadanie: stwórz klasyfikator spamu:
   - Pobierz przykłady spamu i normalnych wiadomości z publicznych zestawów danych aplikacji Apache SpamAssassin (*https://spamassassin.apache.org/old/publiccorpus/*).
   - Wypakuj te dane z archiwów i zapoznaj się z ich strukturą.
   - Rozdziel te dane na zbiory uczący i testowy.
   - Stwórz potok przygotowawczy przekształcający każdą wiadomość w wektor cech. Wiadomości powinny być przekształcane w wektor (rzadki) wskazujący obecność lub nieobecność każdego możliwego słowa. Na przykład, jeśli wszystkie wiadomości zawierają tylko cztery wyrazy: „Hello", „how", „are" i „you", to wiadomość o treści „Hello you Hello Hello you" powinna zostać skonwertowana do postaci wektora [1, 0, 0, 1] (co oznaczałoby, że wyraz „Hello" jest obecny, „how" — nieobecny, „are" — nieobecny i „you" — obecny) lub [3, 0, 0, 2], jeśli wolisz zliczać liczbę wystąpień danego wyrazu.
     Możesz chcieć dodać hiperparametry do potoku przygotowawczego, aby decydować, czy brać pod uwagę nagłówki wiadomości, konwertować rozmiar znaków do jednolitej postaci (np. małe litery), usuwać znaki interpunkcyjne, zastępować wszystkie adresy URL wyrazem „URL", zastępować wszystkie liczby wyrazem „LICZBA", a nawet żeby przeprowadzać **analizę słowotwórczą** (np. usuwać końcówki wyrazów; dostępne są odpowiednie biblioteki środowiska Python),
     Na koniec wypróbuj kilka różnych klasyfikatorów i sprawdź, czy jesteś w stanie stworzyć wydajny filtr spamu (cechujący się zarówno dużą precyzją, jak i pełnością).

Rozwiązania tych ćwiczeń znajdziesz w notatnikach Jupyter dostępnych na stronie *ftp://ftp.helion.pl/przyklady/uczem2.zip*.

[5] Możesz użyć funkcji `shift()` stanowiącej część modułu `scipy.ndimage.interpolation`. Przykładowo, funkcja `shift(image, [2, 1], cval=0)` przesuwa obraz o dwa piksele w dół i jeden piksel w prawo.